EEE 443/543 FALL 2025-26
Assignment # 4

Assignment # 4 : In this project we will consider handwritten character recognition by using a multilayer structure and MNIST dataset. MNIST contains total of 70.000 handwritten characters, each has 28×28 pixel (grayscale) image. 60.000 of these are reserved as training set and the remaining 10.000 are reserved as test set. You may access the dataset through :

http://yann.lecun.com/

https://github.com/mkolod/MNIST#

1 : $28 \times 28 = 784$, and the first thing to do is to convert this image to a vector of length 784. This is called flattening and simple python, numpy codes can do that. Note that due to grayscale, each pixel has a value between 0-255. To speed up the learning process, the input vectors are usually normalized (simple way is to divide each entry by 255).

2 : For classification, we will use a 5 layered CNN structure, inspired by LeNet network structure, see the next 2 slides.

• Input is 28×28 image (This is conceptual. In your code the flattened version will be used)

• If the convolution kernel includes a threshold, a threshold entry will be added to the input, otherwise not.

• First layer H1 is a convolution layer, which contains 4 different 5×5 filter kernels. The output of this convolution layer is 4 different 24×24 images. Note that this layer can be represented as a standard perceptron layer with special and shared weight arrangement, see Lectures on CNN, slide 84. We have two different cases here : filter kernels without threshold and with threshold.

• Convolution kernel without threshold is as explained in the notes. Each filter has $5 \times 5 = 25$ trainable parameters, which are shared in teedforward representation, see slide 84.

• Convolution kernel with a threshold is a simple extension. This time each filter has $5 \times 5 + 1 = 26$ trainable parameters, which are shared in feedforward representation. If without threshold, the (i,j) pixel value is y_{ij} after convolution, with the threshold θ it will be $y_{ij} - \theta$.

• Second layer H2 is 2×2 average pooling layer. See CNN lecture notes, slide 89. Hence its outputs are 4 different downscaled 12×12 images. This layer can be represented as a standard perceptron layer with special and shared weights. But this layer weights are fixed and does not have a trainable parameter. Hence use learning coefficient $\eta = 0$ in this layer.

• Third layer H3 is a convolution layer, which contains 3 different 5×5 filter kernels. Each filter is applied to 4 images at the output of layer H2 separately. Hence the output is $4 \times 3 = 12$ different 8×8 images. This layer can be represented as a standard perceptron layer with special and shared weight arrangement as explained for the layer H1.

• As explained for layer H1, here we also have two options: kernels with or kernels without threshold.

• Fourth layer H4 is 2×2 average pooling layer. Hence its outputs are 12 different downscaled 4×4 images. This layer can be represented as a standard perceptron layer with special and shared weights. But this layer weights are fixed and does not have a trainable parameter. Hence use learning coefficient $\eta = 0$ in this layer.

• Fifth layer H5 is a fully connected layer with a threshold, having 10 output neuron, where each neuron corresponds to a digit that input pattern represents. If the network is trained correctly, the output neuron having the maximum value should represent the digit represented by the input image.

Handwritten Digit Recognition with a Back-Propagation Network

Y. Le Cun, B. Boser, J. S. Denker, D. Henderson,
R. E. Howard, W. Hubbard, and L. D. Jackel
AT&T Bell Laboratories, Holmdel, N. J. 07733

ABSTRACT

We present an application of back-propagation networks to handwritten digit recognition. Minimal preprocessing of the data was required, but architecture of the network was highly constrained and specifically designed for the task. The input of the network consists of normalized images of isolated digits. The method has 1% error rate and about a 9% reject rate on zipcode digits provided by the U.S. Postal Service.

https://abaj.ai/doc/papers/lecun1989handwritten.pdf

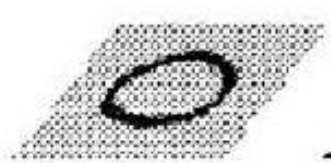
10 OUTPUT

12@4x4 H4

12@8x8 H3

4@12x12 H2

4@24x24 H1



28x28 INPUT

3 : Layers H1-H4 are linear, i.e the activation function is identity mapping. For output layer nonlinear activation function, we will use two cases :

Case 1 : $\sigma(v) = \frac{1}{1+exp(-v)}$ and as cost we use classical average mean squared error, see assignment 3, step **5**.

Case 2 : output activation is softmax function and the cost is categorical cross entropy, see lecture notes on cross entropy, slides 55-57.

4 : Preassign the output neurons to digits as follows : first output neuron should correspond to digit 1, second one to digit 2, so on. The 10th output neuron should correspond to digit 0.

Desired outputs should be chosen as follows. For digit i, only the corresponding output neuron should be 1, all others should be 0, $i = 1, 2, \ldots, 10$.

5 : Choose the learning coefficient η as $\eta = 0.01$.

6 : For the initialization of weights (including the filter kernels), use random initialization, e.g. uniform distribution in a small interval around 0, such as $[-0.01, 0.01]$, or gaussian distribution with zero mean and small variance, such as $\sigma = 0.1$.

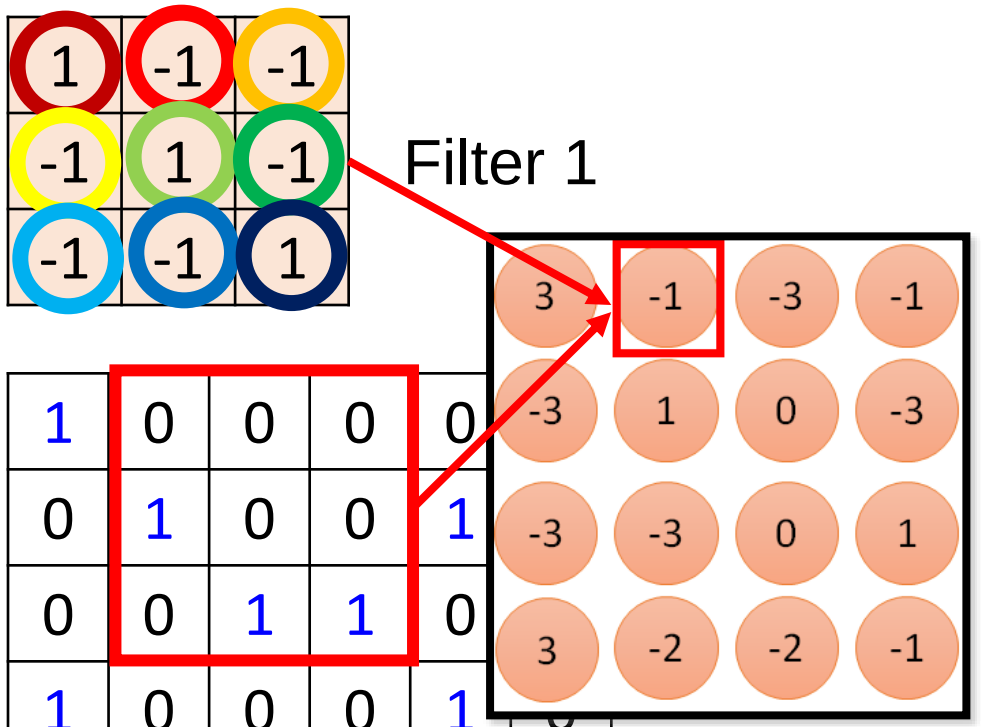
7 : For each of the cases given **2** and **3**, (i.e. Case 1 with or without kernel thresholds, similarly for Case 2), run the back propagation algorithm, online. Note that we have $2 \times 2\times = 4$ runs. Note that in layers H2 and H4, the weights are fixed and will not be updated. For convolutional layers, you have to update the kernel weights. Note that kernel weights are shared, see lecture notes on autoencoders, slide 66. For stopping condition, you may choose an upper bound on the number of epochs, typically 50-100 epochs would suffice. You may also monitor the training set error, and if the decay becomes slower you may stop the training. If the algorithm seems not converging, you may use the hint given in the previous assignment. (For statistically meaningful results, one should have as many runs as possible, use the average success rates and look at the variances, etc.)

8 : After the convergence, give as much relevant data as possible. Record the average error in the training set, the success rate in the training set, how many misclassified patterns you have in the training for each class, number of epochs till convergence, average CPU time per epoch and the total duration spent for the training. Also record the average error in the test set, the success rate in the test set, how many misclassified patterns you have in the test set for each class, the total time it takes to classify a pattern (this is called inference time). You must show your efforts to obtain decent success rates in test (e.g. higher than 90 %). Very poor network performance will also result in a lower grade.

9 : During testing, as in slide 3 and last slide here, give a sample of images generated by the convolution and pooling layer outputs. Use different numerals for each one of the 4 cases indicated in **7**.

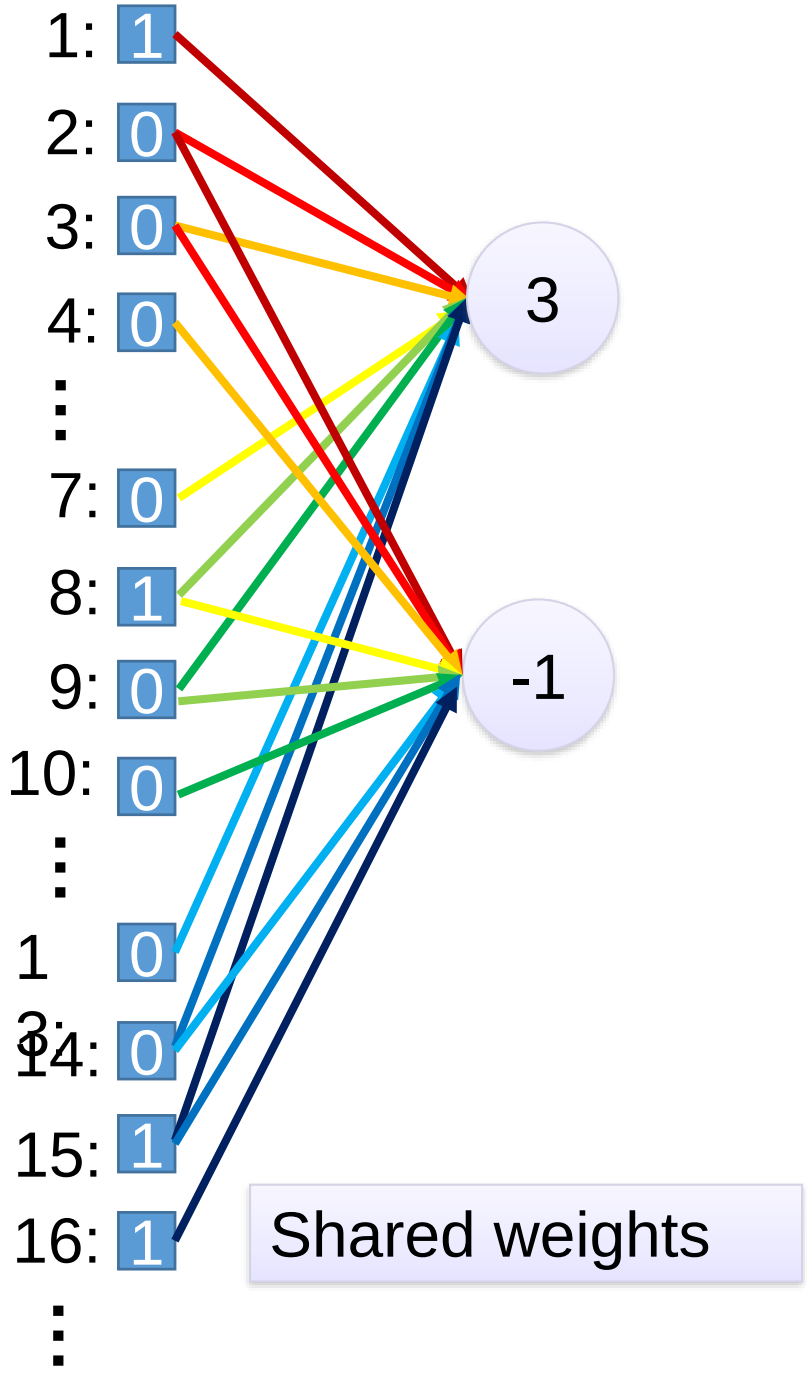
10 : After the step **8**, record the configuration which gives you the best test accuracy result. For this configuration **only**, run the back propagation algorithm by using batch learning with batch size $B = 10,\ 100$. Note that this will require 2 extra simulations. Record the same information as requested in step **8- 9**.

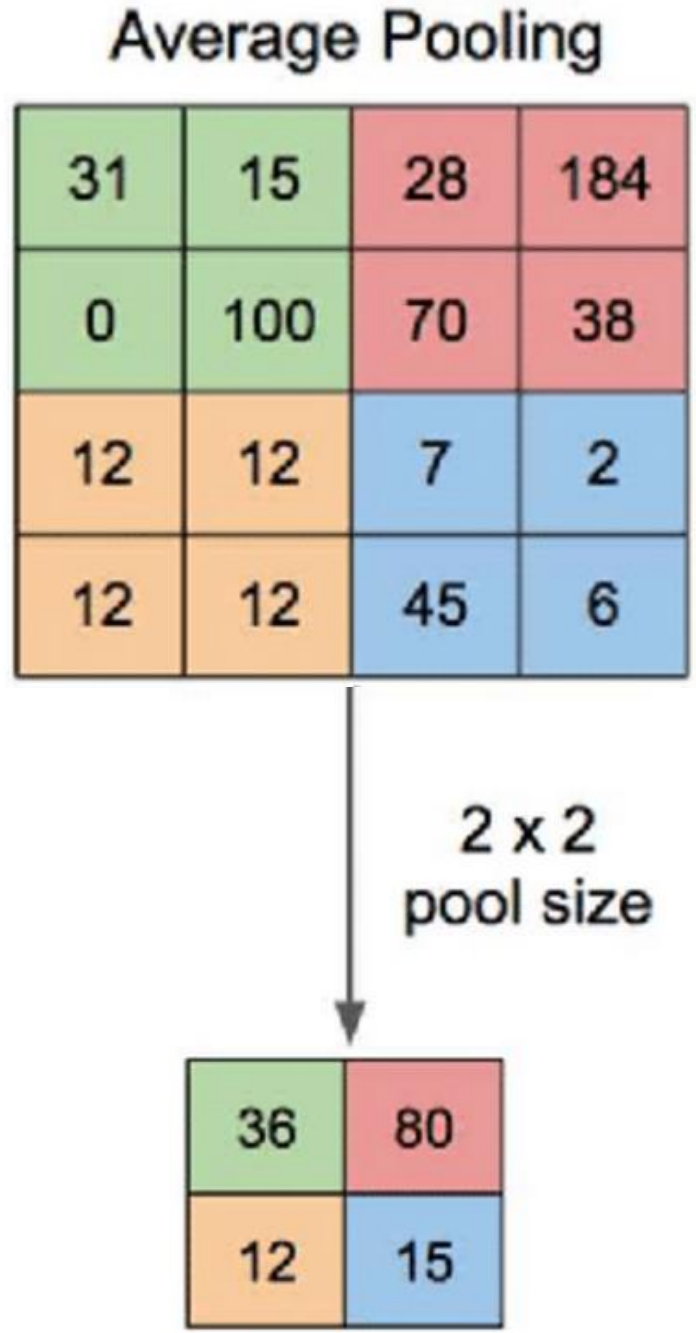
11 : Write your results in a nice report format (in pdf).



Fewer parameters

Even fewer parameters




Average Pooling
31
15
28
184
0
100
70
38
12
12
7
2
12
12
45
6
2 x 2
pool size
36
80
12
15

Backpropagation with weight constraints

- It's easy to modify the backpropagation algorithm to incorporate linear constraints between the weights.

$$To\ constrain: \quad w_1 = w_2$$
$$we\ need: \quad \Delta w_1 = \Delta w_2$$

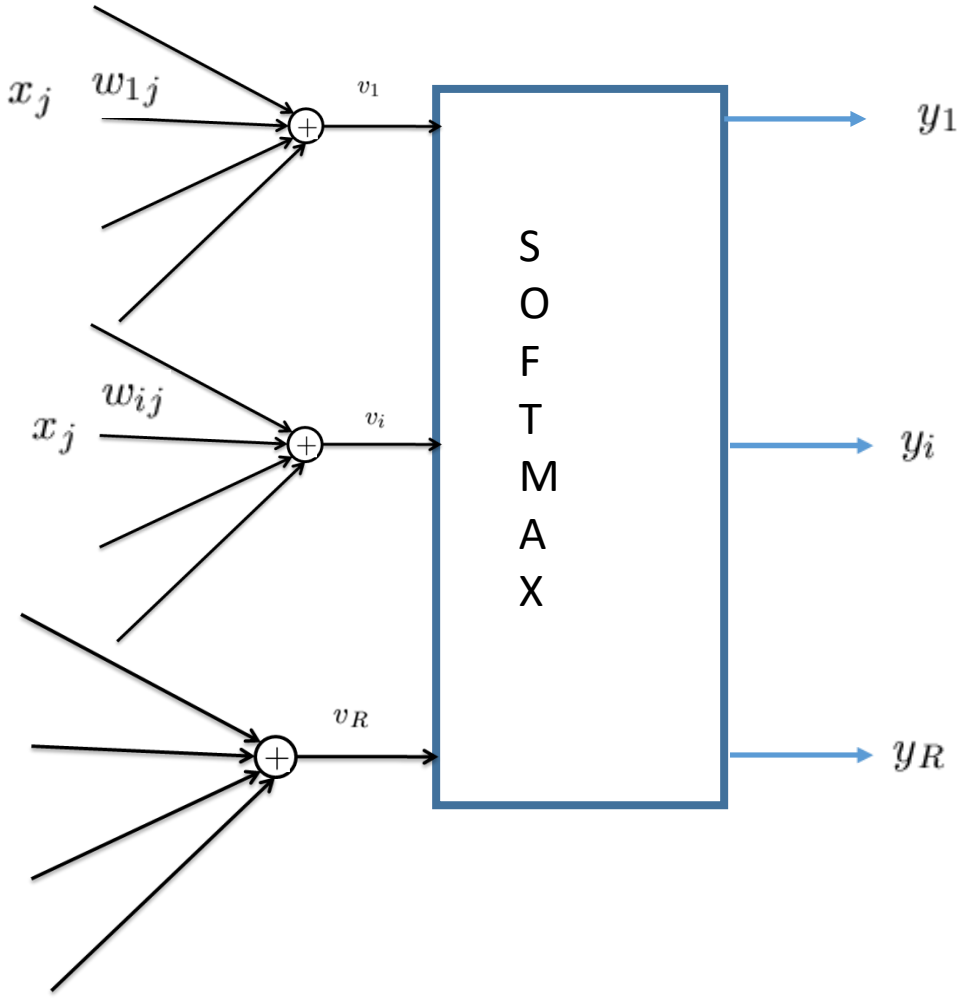
- We compute the gradients as usual, and then modify the gradients so that they satisfy the constraints.

$$compute: \quad \frac{\partial E}{\partial w_1} \quad and \quad \frac{\partial E}{\partial w_2}$$

  - So if the weights started off satisfying the constraints, they will continue to satisfy them.

$$use \quad \frac{\partial E}{\partial w_1} + \frac{\partial E}{\partial w_2} \quad for\ w_1\ and\ w_2$$

https://www.cs.toronto.edu/~hinton/coursera/lecture15/lec15.pdf



$$y_i = \frac{e^{v_i}}{e^{v}1 + \ldots + e^{v_i} + \ldots + e^{v}R}$$

Assume that $x \in Class\ i \rightarrow E = -\log y_i$

$$w_{1j} \leftarrow w_{1j} - \eta \frac{\partial E}{\partial w_{1j}}$$

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial E}{\partial w_{ij}}$$

$$\frac{\partial E}{\partial w_{1j}} = \frac{\partial E}{\partial y_i}\frac{\partial y_i}{\partial v_1}\frac{\partial v_1}{\partial w_{1j}}$$

$$\frac{\partial E}{\partial y_i} = -\frac{1}{y_i}$$

$$\frac{\partial y_i}{\partial v_1} = -\frac{e^{v_1}e^{v_i}}{(\sum_j e^{v_y})^2} = -y_i y_1$$

$$\frac{\partial E}{\partial w_{1j}} = \frac{1}{y_i} y_i y_1 x_j = y_1 x_j$$

$$w_{1j} \leftarrow w_{1j} \leftarrow w_{1j} - \eta y_1 x_j$$

$$y_i = \frac{e^{v_i}}{e^{v_1} + \ldots + e^{v_i} + \ldots + e^{v_R}}$$

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_i}\frac{\partial y_i}{\partial v_i}\frac{\partial v_i}{\partial w_{ij}}$$

$$\frac{\partial E}{\partial y_i} = -\frac{1}{y_i}$$

$$\frac{\partial y_i}{\partial v_i} = \frac{e^{v_i}(\sum_j e^{v_y}) - e^{v_i}e^{v_i}}{(\sum_j e^{v_y})^2} = y_i - y_i^2$$

$$\frac{\partial E}{\partial w_{1j}} = -\frac{1}{y_i} y_i (1 - y_i) x_j = -(1 - y_i) x_j$$

$$w_{1j} \leftarrow w_{1j} \leftarrow w_{1j} + \eta(1 - y_i) x_j$$

Putting it all together

- ex: digit classification (ReLU layer not shown)